Week 10 Summer Homework: Convex Optimization and Smart-Grid Dispatch

Goal: translate a real decision into variables, an objective function, constraints, and verified Python results.

AI policy for this homework. You may use AI or web search to learn syntax, understand official documentation, debug error messages, and ask conceptual questions. You may **not** ask AI to write the required objective function, constraint function, scenario loop, or final explanation for you. Your submitted code must be your own manual programming work. If you use AI for explanation or debugging, write one sentence describing what it helped you understand.

Mathematical reminder. An optimization problem chooses decision variables x to make an objective function as good as possible while satisfying constraints:

$$\begin{aligned} \min_{x} \quad & f(x) \\ \text{subject to} \quad & g_i(x) \le 0, \\ & h_j(x) = 0. \end{aligned}$$

For a simple generator-dispatch problem, the decision variables are generator outputs

$$p = [p_{\text{solar}}, p_{\text{wind}}, p_{\text{gas}}, p_{\text{peaker}}],$$

the objective is total cost, and the most important equality constraint is power balance:

$$p_{\text{solar}} + p_{\text{wind}} + p_{\text{gas}} + p_{\text{peaker}} = \text{demand}.$$

Monday: Translate the Dispatch Story Into Math

Use the generator data below.

Generator	Minimum MW	Maximum MW	Cost \$/MWh
Solar farm	0	90	5
Wind farm	0	120	8
Gas turbine	40	220	45
Backup peaker	0	160	95

Write a short math model for demand $D = 350$ MW.

1. Define the four decision variables in words and symbols.
2. Write the total-cost objective function.
3. Write the power-balance equality constraint.
4. Write the lower and upper bound constraints for all four generators.
5. Predict, without Python, which generators should be used first and why.

Tuesday: Complete the Objective Function in Python

Create a Python file named `week10_dispatch.py`. Start from this skeleton and manually fill the `TODO` sections.

```
import numpy as np
from scipy.optimize import minimize

names = ["solar", "wind", "gas", "peaker"]
cost = np.array([5, 8, 45, 95], dtype=float)
bounds = [(0, 90), (0, 120), (40, 220), (0, 160)]
demand = 350.0

def total_cost(p):
    # TODO: return one number: total generation cost
    # Hint: multiply each generator output by its cost, then add.
    pass
```

1. Implement `total_cost(p)`.
2. Test your function on `p = np.array([90, 120, 140, 0])`.

3. Print the result and show the hand calculation that matches it.

4. Explain why the objective function must return one scalar number, not a list or array.

Manual coding rule. Do not use AI to write the function. You may use `np.sum`, array multiplication, a manually written loop, or ordinary arithmetic.

Wednesday: Complete the Power-Balance Constraint

Extend your script with the equality constraint. Manually fill the `TODO` section.

```
def power_balance(p):
    # TODO: return zero when total generation equals demand
    pass

constraints = [
    {"type": "eq", "fun": power_balance}
]

x0 = np.array([50, 80, 180, 40], dtype=float)

result = minimize(
    total_cost,
    x0,
    method="SLSQP",
    bounds=bounds,
    constraints=constraints,
)

print("success:", result.success)
print("message:", result.message)
print("solution:", result.x)
print("cost:", total_cost(result.x))
```

1. Implement `power_balance(p)`.

2. Run the optimizer for $D = 350$ MW.

3. Submit the printed output.

4. In two or three sentences, explain whether the solution matches your Monday prediction.

Important sign convention. For an equality constraint in `scipy.optimize`, your function should return 0 when the rule is satisfied. For example, total generation minus demand should equal zero.

Thursday: Verify and Stress-Test the Optimizer

Add verification code after the optimizer result. Your code should not simply trust `result.success`; it should check the engineering meaning of the answer.

1. Print total generation, demand, and balance error.

2. For each generator, print whether its output is inside its lower and upper bounds.

3. Re-run the model for at least three demand values: 250 MW, 350 MW, and 450 MW.

4. Create a small table showing demand, solution vector, total cost, and whether the peaker generator is used.

5. Explain when the expensive peaker starts being used and why.

Hints, not code: A `for` loop can run the same optimization for multiple demand values. Be careful that the `power_balance` function uses the current demand value for that run.

Friday: Feynman Technique Post

Write a beginner-friendly post explaining convex optimization using either the food-budget example or the power-grid dispatch example.

1. Explain decision variables as "the numbers we are allowed to choose."

2. Explain the objective function as “the score we want to improve.”
3. Explain constraints as “rules the answer must obey.”
4. Explain convexity using the “lowest point in a valley” analogy.
5. Include one small table, diagram, screenshot, or hand-drawn picture from your work.
6. End with one smart-grid application: economic dispatch, battery scheduling, demand response, or renewable planning.

Submission checklist: Monday math model; Tuesday objective-function script and hand calculation; Wednesday power-balance constraint and optimizer output; Thursday verification code and three demand scenarios; Friday Feynman note.